

AI TRAINING – WEEK 2

DAY 2 ASSIGNMENT

Dictionaries, Nested Dictionaries, Counter & Comprehensions

Learn → Practice → Build → Explain

1. What You Will Learn

You will learn how dictionaries store data using key-value pairs. You will also practice nested dictionaries, Counter, and comprehensions.

- Dictionaries – keys, values, accessing and updating data
- Dictionary methods
- Nested dictionaries
- Counter for counting repeated values
- List and dictionary comprehensions
- Common errors and how to fix them
- Build student, contact, and word-frequency analyzers

2. Student Deliverable – Keep It Simple

Submit ONE Jupyter Notebook only.

File name: Week2_Day2_Dictionaries_YourName.ipynb

Your notebook should contain:

- Part A – Dictionary basics
- Part B – Dictionary methods
- Part C – Nested dictionaries
- Part D – Counter
- Part E – Comprehensions
- Part F – Debugging
- Part G – Student Analyzer
- Part H – Contact Analyzer
- Part I – Word Frequency Analyzer
- Part J – Explain what you learned

No separate Python files, Word/PDF files, or screenshots are required.

3. Part A – Dictionary Basics

```
student = {"name": "Rahul", "age": 21, "course": "Python", "marks": 85}
```

- Print the complete dictionary.

```
Ans:print(student)
```

- Print the student's name.

```
Ans:print(student["name"])
```

- Print the student's marks.

```
Ans:print(student["marks"])
```

- Change the marks to 90.

```
Ans:student["marks"] = 90
```

- Add a city key.

```
Ans:student["city"]="Nellore"
```

- Remove the age key.

```
Ans: del student["age"]
```

- Print the final dictionary.

```
Ans:print(student)
```

```
OUTPUT: {'name': 'Rahul', 'age': 21, 'course': 'Python', 'marks': 85}
```

```
Rahul:85
```

```
{'name': 'Rahul', 'age': 21, 'course': 'Python', 'marks': 90, 'city': 'Nellore'}
```

Think: Why is a dictionary useful for storing information about a student?

4. Part B – Dictionary Methods

Practice: keys(), values(), items(), get(), update(), pop()

- Print all keys.

```
Ans:print(student.keys())
```

- Print all values.

```
Ans:print(student.values())
```

- Print all key-value pairs using items().

```
Ans:print(student.items())
```

- Use get() to safely access a key

```
Ans:print(student.get("name"))
```

```
print(student.get("marks")).
```

- Use update() to add or change information.

```
Ans:student.update({"marks": 90})
```

- Use pop() to remove one item.

```
Ans:student.pop("age")
```

5. Part C – Nested Dictionaries

```
students = {  
    "student1": {"name": "Rahul", "marks": 85},  
    "student2": {"name": "Priya", "marks": 92},  
    "student3": {"name": "Arjun", "marks": 78}  
}
```

- Print the complete dictionary.

```
Ans:print(students)
```

- Print Priya's name.

```
Ans:print(students["student2"]["name"])
```

- Print Priya's marks

```
Ans:print(students["student2"]["marks"])
```

- Change Arjun's marks to 82.

```
Ans:students["student3"]["marks"] =82
```

- Add a city for each student.

```
Ans:students["student1"]["city"] ="ongole"
```

```
students["student2"]["city"]="nellore"
```

```
students["student3"]["city"]="kaluvayi"
```

- Print each student's name and marks using a loop.

```
Ans:for student in students.values():
```

```
    print(student["name"], student["marks"])
```

OUTPUT:

```
{'student1': {'name': 'Rahul', 'marks': 85}, 'student2': {'name': 'Priya', 'marks': 92}, 'student3': {'name':  
'Arjun', 'marks': 78}}
```

Priya

92

Rahul 85

Priya 92

Arjun 82

Challenge: Find the student with the highest marks.

Code:

```
highest_marks = 0
```

```
highest_student = ""
```

```
for student in students.values():
```

```
    if student["marks"] > highest_marks:
```

```
        highest_marks = student["marks"]
```

```
        highest_student = student["name"]
```

```
print(highest_student)
```

```
print( highest_marks)
```

OUTPUT:

priya

92

6. Part D – Counter

Counter is available from the collections module and is useful for counting repeated values.

```
from collections import Counter
```

```
fruits = ["apple", "banana", "apple", "orange", "banana", "apple"]
```

- Create a Counter for fruits.

```
Ans:fruit_count = Counter(fruits)
```

- Find the count of apple.

```
Ans:print(fruit_count["apple"])
```

- Find the count of banana.

```
Ans:print(fruit_count["banana"])
```

- Find the most common fruit.

```
Ans:print(fruit_count.most_common(1))
```

- Try Counter with the string "programming".

```
Ans:text = "programming"
```

```
letter_count = Counter(text)
```

```
print(letter_count)
```

OUTPUT:

```
Counter({'apple': 3, 'banana': 2, 'orange': 1})
```

```
3
```

```
2
```

```
[('apple', 3)]
```

```
Counter({'r': 2, 'g': 2, 'm': 2, 'p': 1, 'o': 1, 'a': 1, 'i': 1, 'n': 1})
```

7. Part E – Comprehensions

Comprehensions provide a short way to create collections.

Example:

```
numbers = [1, 2, 3, 4, 5]
```

```
squares = [n * n for n in numbers]
```

- Create a list of squares from 1 to 10.

```
Ans:squares = [n*n for n in range(1, 11)]
```

- Create a list containing only even numbers from 1 to 20.

```
Ans:even_number = [ n for n in range(1, 21) if n % 2 ==0]
```

- Create a dictionary where numbers 1–5 are keys and their squares are values.

```
Ans:squares = {n: n * n for n in range(1, 6)}
```

- Create a dictionary of student names and marks only for students who scored 80 or above.

```
Ans:students = {
```

```
    "Rahul": 85,
```

```
    "Priya": 92,
```

```
    "Arjun": 78,
```

```
    "Anita": 88,
```

```
    "Ravi": 65
```

```
}
```

```
top_students = {
```

```

name: marks

for name, marks in students.items()

    if marks >= 80

}

print(top_students)

```

OUTPUT:

```
{'Rahul': 85, 'Priya': 92, 'Anita': 88}
```

8. Part F – Quick Debugging

Find the error, fix it, run the code, and explain the mistake.

Debugging Problem 1

```

student = {"name": "Rahul", "age": 21}
print(student["marks"])

code:student = {"name": "Rahul", "age": 21}

    student["marks"] = 85

    print(student["marks"])

```

output:85

Explanation:keyerror="marks"

The dictionary does not have a "marks" key.

Debugging Problem 2

```

student = {"name": "Rahul"}
print(student[0])

code:student = {"name": "Rahul"}

    print(student["name"])

```

output:Rahul

Explanation:KeyError: 0,A dictionary is accessed using keys, not positions/index numbers

0 is an index, but dictionaries use keys such as "name".

Debugging Problem 3

```

student = {"name": "Rahul"}
student.add("city", "Hyderabad")

code:student = {"name": "Rahul"}

    student["city"] = "Hyderabad"

```

Output:Rahul

Explanation: Error because Dictionaries do not have an .add() method.

Debugging Problem 4

```
numbers = [1, 2, 3, 4]
squares = {n: n * n for n in numbers]
print(squares)

code: numbers = [1, 2, 3, 4]

    squares = {n: n * n for n in numbers}

    print(squares)
```

output:{1: 1, 2: 4, 3: 9, 4: 16}

Explanation: Dictionary comprehension uses { } brackets. We used] instead of } to close the dictionary, so a SyntaxError occurred.

9. Part G – Hands-on Activity: Student Analyzer

Start with: student = {"name": "Rahul", "age": 21, "course": "Python", "marks": 85}

Your program should:

- Display student details.

```
Ans: student = {

    "name" : "Iasya",

    "age": 20,

    "course": "python",

    "marks": 85

}
```

```
print("Student Details:")

print( student["name"])

print(student["age"])

print(student["course"])

print(student["marks"])
```

- Check whether the student passed or failed.

Ans: if student["marks"] >=40:

```
    print("Result: pass")
```

else:

```
print("Result: Fail")
```

- Assign a grade based on marks.

Ans:if student["marks"] >= 90:

```
    grade = "A+"
```

elif student["marks"] >= 80:

```
    grade = "A"
```

elif student["marks"] >= 70:

```
    grade = "B"
```

elif student["marks"] >= 60:

```
    grade = "C"
```

elif student["marks"] >= 40:

```
    grade = "D"
```

else:

```
    grade ="F"
```

```
student["grade"] = grade
```

- Allow marks to be updated.

Ans:student["marks"] = 90

- Display the final student information

Ans:print("\nFinal Student Information:")

```
print(student)
```

OUTPUT:

Student Details:

lasya

20

python

85

```
{'student1': {'name': 'Rahul', 'marks': 85, 'city': 'ongole'}, 'student2': {'name': 'Priya', 'marks': 92, 'city': 'nellore', 'student3': {'name': 'Arjun', 'marks': 82, 'city': 'kaluvayi'}}
```

10. Part H – Hands-on Activity: Contact Analyzer

```
contacts = {"Rahul": "9876543210", "Priya": "9123456780", "Arjun": "9988776655"}
```

- Add a new contact.

```
Ans:contacts["Yasaswini"] = "9014988999"
```

- Search for a contact by name.

```
Ans:name = "Priya"
```

```
    if name in contacts:
```

```
        print(contacts[name])
```

```
    else:
```

```
        print("Contact not found")
```

- Update a phone number.

```
Ans: contacts["Rahul"] = "9898787849"
```

- Delete a contact.

```
Ans:del contacts["Priya"]
```

- Display all contacts.

```
Ans:print("\nAll Contacts:")
```

```
    for name, phone in contacts.items():
```

```
        print(name, ":", phone)
```

OUTPUT:

Phone: 9123456780

All Contacts:

Rahul : 9898787849

Arjun : 9988776655

Yasaswini : 9014988999

Bonus: Store phone, email, and city for each contact using a nested dictionary.

```
code:contacts = {
```

```
    "lasi": {
```

```
        "phone": "9878328089",
```

```
        "email": "lasi@gmail.com",
```

```
        "city": "nellore"
```

```

    },
    "pavithra": {
        "phone": "9123456709",
        "email": "pavithra@gmail.com",
        "city": "chennai"
    },
    "yashu": {
        "phone": "9098783409",
        "email": "yashu@gmail.com",
        "city": "kandhukur"
    }
}

contacts["ishu"] = {
    "phone": "9876501234",
    "email": "ishu@gmail.com",
    "city": "Chennai"
}

# Search for a contact
name = "Pavithra"

if name in contacts:
    print( contacts[name]["phone"])
    print(contacts[name]["email"])
    print(contacts[name]["city"])
else:
    print("Contact not found")

# Update phone number
contacts["lasi"]["phone"] = "8089328798"

```

```
# Delete a contact
```

```
del contacts["lasi"]
```

```
# Display all contacts
```

```
print("\nAll Contacts:")
```

```
for name, details in contacts.items():
```

```
    print(name)
```

```
    print(details["phone"])
```

```
    print(details["email"])
```

```
    print( details["city"])
```

```
    print()
```

Output:

All Contacts:

output 1:

pavithra

9123456709

pavithra@gmail.com

chennai

output 2:

yashu

9098783409

yashu@gmail.com

kandhukur

output 3:

ishu

9876501234

ishu@gmail.com

Chennai

11. Part I – Hands-on Activity: Word Frequency Analyzer

Example sentence: "python is easy and python is powerful"

- Convert the sentence into words.
- Use Counter to count each word.
- Display the word counts.
- Find the most frequently used word.
- Bonus: convert all words to lowercase before counting.

Code:

```
from collections import Counter

sentence = "python is easy and python is powerful"

words = sentence.split()

words = [word.lower() for word in words]

word_count = Counter(words)

print("Word counts:", word_count)

most_common_word = word_count.most_common(1)

print("Most frequently used word:", most_common_word[0][0])

print("Count:", most_common_word[0][1])
```

OUTPUT:

Word counts: Counssster({'python': 2, 'is': 2, 'easy': 1, 'and': 1, 'powerful': 1})

Most frequently used word: python

Count: 2

12. Part J – Explain in Your Own Words

1. What is a dictionary?

Answer: A dictionary is a Python collection that stores data in key-value pairs.

2. What is a key-value pair?

Answer: A key-value pair is a key and its corresponding value stored together in a dictionary.

3. Why are dictionaries useful?

Answer: Dictionaries are useful for storing and accessing data using key-value pairs.

4. What is a nested dictionary?

Answer: A nested dictionary is a dictionary inside another dictionary.

5. Why is Counter useful?

Answer: Counter is useful for counting how many times each item appears in a collection.

6. What is a comprehension?

Answer: A comprehension is a short and simple way to create a list, set, or dictionary using a loop.

7. What is the difference between a normal loop and a comprehension?

Answer: A normal loop uses multiple lines of code, while a comprehension uses a shorter and simpler way to create a collection.

8. When would you use a dictionary instead of a list?

Answer: We use a dictionary when we want to store and access data using keys and values.

13. Student Submission Checklist

- I practiced dictionary creation and access.✓
- I used dictionary methods.✓
- I practiced nested dictionaries.✓
- I used Counter.✓
- I practiced comprehensions.✓
- I fixed the debugging problems✓.
- I completed the Student Analyzer.✓
- I completed the Contact Analyzer.✓
- I completed the Word Frequency Analyzer.✓
- I explained the concepts in my own words.✓
- My notebook runs without major errors.✓

14. After This Assignment, You Should Be Able To

- Store structured information using dictionaries.
- Access and update dictionary values.
- Work with nested dictionaries.
- Count repeated values using Counter.
- Create lists and dictionaries using comprehensions.
- Choose dictionaries when data is naturally represented as key-value pairs.
- Build small real-world data analyzers.

Key Principle: Don't just learn dictionary syntax. Learn how to organize, search, update, and analyze data.